

Level 1 – Understanding

Q1. For each column, state the data type and reason.

1. PAN number — CHAR(10): PAN always has exactly 10 characters.
2. Interest amount — NUMERIC(15,2): gives exact decimal values for money.
3. Number of days — SMALLINT: 3,650 is a small whole number.
4. UPI authorised time — TIMESTAMP: stores date and exact time.
5. Loan instalment due — DATE: only the date is needed.
6. Branch complaint note — TEXT: stores variable-length text.
7. Transaction counter — BIGINT: can store a very large number.
8. Auto-renew — BOOLEAN: stores true or false.

Q2. A developer proposes FLOAT for interest_amount.

```
SELECT 0.1::FLOAT + 0.2::FLOAT;  
SELECT 0.1::NUMERIC + 0.2::NUMERIC;
```

Expected result: FLOAT can show 0.30000000000000004, while NUMERIC gives 0.3.

Business impact: FLOAT can cause small rounding errors in interest and other financial calculations.

Q3. Complete the constraint table.

NOT NULL: Value cannot be empty. Example: customer name.

UNIQUE: Values cannot repeat. Example: account number.

PRIMARY KEY: Uniquely identifies each row and cannot be NULL. Example: customer ID.

FOREIGN KEY: Value must exist in another table. Example: customer ID in account.

CHECK: Value must satisfy a condition. Example: amount >= 500.

DEFAULT: Gives a value automatically when none is supplied. Example: status = ACTIVE.

Q4. State two differences between PRIMARY KEY and UNIQUE, and explain the choice of deposit_id.

A table can have only one PRIMARY KEY, but it can have multiple UNIQUE constraints. A PRIMARY KEY cannot contain NULL, while a UNIQUE column can generally allow NULL.

deposit_id is a stable internal identifier, while receipt_number is a business value.

Q5. Explain DELETE, TRUNCATE and DROP.

DELETE: Removes selected rows, supports WHERE, and is slower on large tables.

TRUNCATE: Removes all rows, does not use WHERE, and is usually faster.

DROP: Removes the table itself, including its structure.

In PostgreSQL, these operations can be rolled back when used inside a transaction. DELETE and TRUNCATE fire their respective table-level triggers; DROP does not fire row-level DELETE triggers.

Q6. The bank must keep seven years of transaction history. What is wrong with deleting records older than 90 days?

The query deletes data that must be retained for seven years. The bank should keep the required history and use an archive/retention process only after the required retention period.

Q7. Predict the results involving NULL.

```
SELECT NULL = NULL;
```

```

SELECT NULL <> NULL;
SELECT 5000 + NULL;
SELECT COALESCE(NULL, NULL, 'FALLBACK');
SELECT 'A' || NULL || 'B';

```

Results: NULL, NULL, NULL, FALLBACK, NULL.

Q8. Why do the two branch queries return different row counts?

AND has higher precedence than OR. The first query means `branch_id = 1 OR (branch_id = 2 AND principal_amount > 500000)`. The second applies the amount condition to both branches because of the brackets.

Q9. Correct the quarterly date filter.

If `txn_date` is a timestamp, `BETWEEN` with `DATE '2026-03-31'` only reaches midnight at the start of that day. Use: `txn_date >= DATE '2026-01-01' AND txn_date < DATE '2026-04-01'`.

Q10. Why can a SELECT alias be used in ORDER BY but not WHERE?

SQL logically evaluates `WHERE` before `SELECT`, so the alias does not exist yet. `ORDER BY` is evaluated later, so it can use the `SELECT` alias.

Level 2 – Applying SQL

Q11. Create the recurring_deposit table with named constraints.

```

CREATE TABLE recurring_deposit (
    rd_id BIGINT CONSTRAINT pk_recurring_deposit PRIMARY KEY,
    customer_id BIGINT CONSTRAINT fk_rd_customer REFERENCES customer(customer_id),
    branch_id INT CONSTRAINT fk_rd_branch REFERENCES branch(branch_id),
    rd_number VARCHAR(20) CONSTRAINT uq_rd_number UNIQUE,
    monthly_amount NUMERIC(15,2) CONSTRAINT chk_rd_monthly_amount CHECK
(monthly_amount >= 500),
    interest_rate NUMERIC(5,2) CONSTRAINT chk_rd_interest_rate CHECK
(interest_rate > 0 AND interest_rate <= 12),
    tenure_months INT CONSTRAINT chk_rd_tenure CHECK
(tenure_months BETWEEN 6 AND 120),
    start_date DATE CONSTRAINT nn_rd_start_date NOT NULL DEFAULT
CURRENT_DATE,
    maturity_date DATE CONSTRAINT nn_rd_maturity_date NOT NULL,
    installments_paid INT CONSTRAINT chk_rd_installments CHECK (installments_paid >= 0) DEFAULT 0,
    status VARCHAR(10) CONSTRAINT chk_rd_status CHECK (status IN
('ACTIVE','MATURED','DEFAULTED','CLOSED')) DEFAULT 'A' CONSTRAINT chk_rd_maturity CHECK (maturity_date
> start_date)
);

```

Note: if your existing customer/branch tables use different column names, use the exact names from your database.

Q12. Write six statements that should be rejected.

```

-- 1
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months, maturity_date)
VALUES (9001, 1001, 1, 'TEST001', 100, 7, 12, '2027-01-01');

-- 2
INSERT INTO recurring_deposit (rd_id, customer_id, branch_id, rd_number, monthly_amount,
interest_rate, tenure_months, maturity_date)
VALUES (9002, 5555, 1, 'TEST002', 1000, 7, 12, '2027-01-01');

-- 3
INSERT INTO recurring_deposit (rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate,
tenure_months, start_date, maturity_date)
VALUES (9003, 1001, 1, 'TEST003', 1000, 7, 12, '2026-06-01', '2026-01-01');

-- 4
INSERT INTO recurring_deposit

```

```
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months, maturity_date)
VALUES (9004, 1001, 1, 'RD2026000001', 1000, 7, 12, '2027-01-01');
```

-- 5

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months, maturity_date, status)
VALUES (9005, 1001, 1, 'TEST005', 1000, 7, 12, '2027-01-01', 'PAUSED');
```

-- 6

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months, maturity_date, installments_p
VALUES (9006, 1001, 1, 'TEST006', 1000, 7, 12, '2027-01-01', -3);
```

Expected errors: CHECK violation, FOREIGN KEY violation, maturity CHECK violation, UNIQUE violation, status CHECK violation, and installments CHECK violation respectively. Record the exact error from your own PostgreSQL run.

Q13. Insert the three recurring deposits in one statement.

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate, tenure_months, start_date, maturity_date)
VALUES
(8001, 1001, 1, 'RD2026000001', 5000, 7.10, 24, '2026-03-01', '2028-03-01'),
(8002, 1005, 5, 'RD2026000002', 2500, 6.85, 12, '2026-03-05', '2027-03-05'),
(8003, 1012, 4, 'RD2026000003', 10000, 7.35, 60, '2026-03-10', '2031-03-10');
```

Expected: 3 rows inserted.

Q14. Add a nullable nominee_name column and set RD 8001's nominee.

```
ALTER TABLE recurring_deposit
ADD COLUMN nominee_name VARCHAR(60);
```

```
UPDATE recurring_deposit
SET nominee_name = 'Meera Reddy'
WHERE rd_id = 8001;
```

Q15. Raise the minimum monthly instalment to 1,000.

```
SELECT *
FROM recurring_deposit
WHERE monthly_amount < 1000;
```

```
ALTER TABLE recurring_deposit
DROP CONSTRAINT chk_rd_monthly_amount;
```

```
ALTER TABLE recurring_deposit
ADD CONSTRAINT chk_rd_monthly_amount
CHECK (monthly_amount >= 1000);
```

First check for violating rows. If any exist, correct them before adding the new constraint.

Q16. Add mandatory scheme_code to a populated table.

```
ALTER TABLE recurring_deposit
ADD COLUMN scheme_code VARCHAR(8) NOT NULL;
```

Expected: error because existing rows have no value for the new NOT NULL column.

```
ALTER TABLE recurring_deposit
ADD COLUMN scheme_code VARCHAR(8) DEFAULT 'RD001'
```

NOT NULL; This succeeds by giving existing rows a value.

Q17. List fixed deposits of 500,000 or more, largest first.

```

SELECT receipt_number, principal_amount, rate, maturity_date
FROM fixed_deposit
WHERE principal_amount >= 500000
ORDER BY principal_amount DESC;

```

Q18. List deposits booked from 1 to 10 March 2026 inclusive.

```

SELECT *
FROM fixed_deposit
WHERE booked_date BETWEEN DATE '2026-03-01' AND DATE '2026-03-10'
ORDER BY booked_date, principal_amount DESC;

```

If booked_date is a timestamp, use >= DATE '2026-03-01' AND < DATE '2026-03-11'.

Q19. Show deposits with no nominee and those whose nominee is not Priya Verma.

```

SELECT *
FROM fixed_deposit
WHERE nominee_name IS NULL;

SELECT *
FROM fixed_deposit
WHERE nominee_name <> 'Priya Verma'
OR nominee_name IS NULL;

```

A simple <> condition does not include NULL because NULL represents an unknown value.

Q20. Show annual interest and top five deposits.

```

SELECT receipt_number, principal_amount,
ROUND(principal_amount * rate / 100, 2) AS annual_interest
FROM fixed_deposit
ORDER BY annual_interest DESC
LIMIT 5;

```

Q21. Show receipt number, customer full name, principal and deposit segment.

```

SELECT fd.receipt_number, CONCAT(c.first_name, ' ',
c.last_name) AS customer_name, fd.principal_amount,
CASE
    WHEN fd.principal_amount >= 1000000 THEN 'HIGH VALUE'
    WHEN fd.principal_amount >= 100000 THEN 'STANDARD'
    ELSE 'SMALL'
END AS segment
FROM fixed_deposit fd
JOIN customer c ON c.customer_id = fd.customer_id
ORDER BY
CASE
    WHEN fd.principal_amount >= 1000000 THEN 1
    WHEN fd.principal_amount >= 100000 THEN 2
    ELSE 3
END;

```

Use the actual customer name column names from your database if they differ.

Q22. Find UPI transactions over 50,000 and give LIMIT syntax.

```

SELECT txn_id, account_id, amount, narration
FROM account_txn
WHERE narration ILIKE '%UPI%'
AND amount > 50000
ORDER BY amount DESC
LIMIT 10;

```

PostgreSQL: LIMIT 10. SQL Server: TOP 10. Oracle: FETCH FIRST 10 ROWS ONLY.

Q23. Safely break deposit 7009 and roll back.

```
BEGIN;

SELECT *
FROM fixed_deposit
WHERE deposit_id = 7009;

UPDATE fixed_deposit
SET status = 'BROKEN',
    auto_renew = 'N'
WHERE deposit_id =
7009;

SELECT *
FROM fixed_deposit
WHERE deposit_id = 7009;

ROLLBACK;
```

Q24. Run the maturity update safely and confirm the count.

```
BEGIN;

SELECT COUNT(*)
FROM fixed_deposit
WHERE status = 'ACTIVE'
    AND maturity_date <= DATE '2026-06-30';

UPDATE fixed_deposit
SET status = 'MATURED'
WHERE status = 'ACTIVE'
    AND maturity_date <= DATE '2026-06-30';

SELECT COUNT(*)
FROM fixed_deposit
WHERE status = 'MATURED'
    AND maturity_date <= DATE '2026-06-30';

ROLLBACK;
```

The number selected before the update should match the number updated.

Q25. Default RD 8002 and then close it without deleting it.

```
UPDATE recurring_deposit
SET status = 'DEFAULTED'
WHERE rd_id = 8002;

UPDATE recurring_deposit
SET status = 'CLOSED'
WHERE rd_id = 8002;
```

Keep the row because banking history is needed for audit and future reference.

Level 3 – Design and Judgement

Q26. Vendor supplies principal_amount as text in different formats.

Do not store the final financial amount as VARCHAR. Use an exact numeric type such as NUMERIC(15,2). Clean and convert the incoming values during the staging/data-ingestion step before loading the main table.

Q27. Choose between ON DELETE CASCADE and RESTRICT for fixed deposits.

I would use RESTRICT. Closing a customer does not mean their deposit history should be deleted. The bank needs the records for audit, regulatory and historical purposes. A customer relationship can be closed while deposit records remain. Therefore, automatic CASCADE deletion is risky for banking history.

Q28. Add employee authorisation to a 40-million-row account_txn table.

authorised_by BIGINT

It should be nullable because authorisation may not apply to every transaction.

It should have a foreign key to employee(employee_id).

For the 40 million existing rows, add the column as NULL initially and populate it where historical authorisation information is available.

Q29. Compare a CHECK constraint with a deposit_status lookup table.

CHECK advantages: simple and easy to understand; no extra table or join is needed.

Lookup table advantages: statuses can be added without changing the main table constraint; extra information can be stored for each status.

For this table, a CHECK constraint is suitable because there are only four fixed statuses.

Q30. What should be done after UPDATE account SET status = 'DORMANT'; ran in production?

1. Stop further changes to the affected table if possible.
2. Check whether the transaction can still be rolled back.
3. Since autocommit is on, check audit/database logs for the previous values.
4. Find exactly how many accounts were affected.
5. Restore the correct values using available audit or backup information.
6. Verify the restored records.
7. Check whether any application process acted on the DORMANT status.
8. Inform the relevant production/database team and document the incident.
9. Require review before production UPDATE or DELETE statements.
10. Use transactions and safer tools that warn about UPDATE statements without a WHERE clause.